

ecalj Install

To install the ecalj package, you need to install software and Python modules used in ecalj. In addition, we make some softlinks for convenience. Most steps are almost automatic. Follow the steps below.

1. Required software and native tools

The core of ecalj is written in modern Fortran (Fortran 90/2003 style). The Fortran source files are:

```
ecalj/SRC/main/*.f90
ecalj/SRC/subroutines/*.f90
```

We can automatically turn on/off the cpp switches for GPU support (OpenACC/CuBLAS) during the compilation below.

Recommended/needed native tools and libraries (Ubuntu/Debian examples follow):

- `git` (to clone/update the ecalj repository). `gitk` is convenient for visual history inspection.
- A Fortran compiler: `gfortran` (GNU), `ifort` (Intel), or `nvfortran` (NVIDIA) — choose one supported by your site. `nvfortran` is for GPU.
- BLAS/LAPACK/FFT libraries. Intel MKL is recommended where available; OpenBLAS is an alternative.
- MPI implementation: Open MPI or MPICH. (Open MPI 4.x worked on Ubuntu 24.04.)
- build tools: `cmake`, `make`, `bash`.
- plotting / utilities: `gnuplot` (optional for plotting scripts and tests).

On Debian/Ubuntu, use the following command to install these tools.:

```
sudo apt update
sudo apt install -y git build-essential cmake make gfortran libopenmpi-
dev openmpi-bin \
    libblas-dev liblapack-dev libfftw3-dev libopenblas-dev gnuplot
```

Notes and tips:

- Intel MKL works well for Intel compiler and gfortran.
- Recently, we may need to install such tools under your home directory.

Example: in my case (ThinkPad T14, Ubuntu 24.04), we use:

- Open MPI 4.1.6
- CMake 3.28.x
- make 4.3

- gfortran 13.x
- Intel MKL 2020.x (where applicable)

`git` helps with upgrades and tracking local changes (e.g., `git diff` at `ecalj/`).

2. Python and Python modules (using pyenv)

[!TIP]

It is recommended to use `pyenv` to manage your Python version for `ecalj`. This keeps your system Python untouched and makes your setup more reproducible. You do not need `pyenv-virtualenv` unless you want to manage multiple environments; for a single `ecalj` environment, `pyenv` alone is sufficient.

Note: Recent Ubuntu and similar systems work best when you keep your project Python environment separate from the system Python.

Python $\geq$ 3.9 is required. Below are the recommended steps to install `pyenv`, set up your shell, install a Python version, and install the required Python packages for `ecalj`.

1. Install prerequisites (Debian/Ubuntu example):

```
sudo apt update
sudo apt install -y build-essential curl libssl-dev zlib1g-dev libbz2-
dev \
    libreadline-dev libsqlite3-dev wget llvm libncurses5-dev libncursesw5-
dev \
    xz-utils tk-dev libffi-dev liblzma-dev python-openssl git
```

1. Install `pyenv` (recommended install method):

```
curl https://pyenv.run | bash
```

1. Add `pyenv` initialization to your shell startup (`~/.bashrc` or `~/.profile`):

```
# pyenv
export PATH="$HOME/.pyenv/bin:$PATH"
eval "$(pyenv init --path)"
eval "$(pyenv init -)"
```

Then reload your shell:

```
source ~/.bashrc
```

1. Install a Python version for ecalj. Example: install latest 3.11 and set it as local version:

```
pyenv install 3.11.6      # choose a concrete, available patch version
pyenv local 3.11.6       # or pyenv global 3.11.6 for system-wide in that
                           account
```

1. Upgrade pip and install required packages:

```
pip install --upgrade pip
pip install numpy pandas seekpath spglib pymatgen mp-api scipy plotly
gnuplot cif2cell
```

Notes:

- If you prefer conda/anaconda, you can substitute that workflow, but `pyenv` keeps system Python untouched and works well with multiple project environments.
- On some systems you may prefer to install Python build dependencies via your package manager (examples above are for Debian/Ubuntu).

Optional: create a `requirements.txt` in your project with the packages above for reproducible installs:

```
numpy
pandas
seekpath
spglib
pymatgen
mp-api
scipy
plotly
gnuplot
cif2cell
```

PROF

Then install with:

```
pip install -r requirements.txt
```

3. Install and InstallTest

For ohtaka and kugui(GPU machine) in ISSP, skip this section and see [installISSP.md](#).

Get ecalj package

The ecalj package is available at <https://github.com/tkotani/ecalj/>.
Check out the master branch.

For ISSP machines, see <https://github.com/msobt/ecaljdoc>.

To clone ecalj:

```
git clone https://github.com/tkotani/ecalj.git
```

After cloning, a directory `ecalj/` appears.
You can check the history of ecalj with:

```
cd ecalj
gitk --all
```

InstallAll.py

To install, use `InstallAll.py [Options]`. For help, run:

```
./InstallAll.py --help
```

Example help output:

```
usage: InstallAll [-h] [-np NP] [--clean] [--gpu] [--bindir BINDIR] --fc
FC [--notest] [--verbose]
```

```
Install ecalj and run tests.
```

```
options:
```

<code>-h, --help</code>	show this help message and exit
<code>-np NP</code>	number of mpi cores for install test default: 8 specify the number of MPI parallelization in test

```
calculation
```

<code>--clean</code>	Clean CMakeCache CMakeFiles before make default: none delete the cache files before compiling
<code>--gpu</code>	nvfortran for GPU default: none compile the GPU and GPU-MP version
<code>--bindir BINDIR</code>	ecalj binaries and scripts
<code>--fc FC</code>	fortran compiler gfortran/ifort/nvfortran
<code>--notest</code>	no test. only compile
<code>--verbose</code>	verbose on for debug

`InstallAll.py` copies all binaries and scripts for ecalj to your directory specified by `--bindir` `foobar/` (default is `$HOME/bin/`). Set `foobar/` as you like. `--fc` is required.

For GPU, use `--fc nvfortran` together with `--gpu`.

at `ecalj/SRC/TestInstall/`. (`testecalj.py` is the script for test)

Install ecalj

`InstallAll.py` compiles and installs ecalj, then runs install tests at `ecalj/SRC/TestInstall/` (see `testecalj.py`).

If successful, you will see 'OK! ALL PASSED!' at the end. The process may take 3 ~ 10 minutes.

To install ecalj, first check the help:

```
./InstallAll.py --help
```

Typical install command:

```
./InstallAll.py --fc gfortran
```

For GPU builds, use:

```
./InstallAll.py --fc nvfortran --gpu
```

To start from scratch, add `--clean`.

This will compile the Fortran source, link, copy all programs and scripts to your bin, and run the minimum install tests at `ecalj/SRC/TestInstall/` (about ~ 5 minutes).

PROF

Tip: Check `./InstallAll.py` for details on where binaries are installed (default: `BINDIR=~/.bin`).

Tools like `getsymbl`, `viewvesta`, and `vasp2ctrl` are softlinked.

If all tests pass, you will see output like:

```
PASSED! TEST 1 out.lmf.copt
PASSED! TEST 1 out.lmf.te
...
PASSED! nio_gwsc/QPU
PASSED! nio_gwsc/log.nio
PASSED! fe_gwsc/QPU
PASSED! fe_gwsc/QPD
PASSED! fe_gwsc/log.fe
PASSED! ni_crpa/Screening_W-v.h
PASSED! ni_crpa/Screening_W-v_crpa.h
```

```
PASSED! srvo3_crpa/Screening_W-v.h
PASSED! srvo3_crpa/Screening_W-v_crpa.h
OK! ALL PASSED ===
    See work/summary.txt

real    4m49.712s
user    19m38.952s
sys     3m38.583s
```

You can also run individual tests at `ecalj/SRC/TestInstall/` with:

```
./testecalj.py si_gwsc
```

- Each test has its own directory (e.g., `si_gwsc`).
- Each test generates `ecalj/SRC/TestInstall/work/si_gwsc`, runs computation, and compares results with reference data in `ecalj/SRC/TestInstall/si_gwsc`.
- Tests are simple and described in `testecalj.py`. It is easy to add your own test.

If installation fails, you may need to restart from clean. Delete `CMakeFiles` and `CMakeCache.txt` at `ecalj/SRC/exec/`. Sometimes, old `*.mod` files under `SRC/` cause issues. Usually, running `./InstallAll.py --clean` is enough to fix this.

If parallel make (`make -j`) fails, run make by hand:

```
cd ecalj/SRC/exec/
rm -rf CMakeFiles CMakeCache.txt
FC=nvfortran cmake .
make
```

—
PROF

After installation, set your command path (BINDIR). For example, add this to your `.bashrc` if you move all ecalj binaries to `~/bin`:

```
PATH="$~/bin/:$PATH"
```

Install VEST and getsymf

It is convenient to view structures with VESTA.

(Example: I installed `VESTA-gtk3.tar.bz2` (ver. 3.5.8, built on Aug 11 2022, 23.8MB) on Ubuntu 24.)

At `ecalj/StructureTool/`, we have the `viewvesta` command. Try:

```
viewvesta ctrl.si
```

to check the structure in the viewer. In `/StructureTool`, we have converters: `vasp2ctrl` and `ctrl2vasp`. These allow you to convert structures with POSCAR.

In addition, you may need to install `getsym1.py` to obtain symmetry lines for band plots.

Generated `sym1.*` files are used for band plots in `ecalj`.

As long as you have `spglib` and `seekpath`, you usually do not need extra steps.

For more details, see the memo: [./GetSym1/README.org](http://GetSym1/README.org).

Additional memo

- When `InstallAll.py` has finished, all binaries and shell scripts are copied to `--bindir foobar` (default: `~/bin/`).
- Clean up:

If something goes wrong, run `InstallAll.py` again with `--clean`.

You may need to do:

```
rm -f CMakefiles CMakeCache.txt
```

at `ecalj/SRC/exec/` (and remove all `*.mod` files under `SRC/`). Usually, you do not need to do this.

- Compile Fortran only:

To compile Fortran source only, move to `ecalj/SRC/exec/` and run:

```
FC=fortran cmake . -D CMAKE_BUILD_TYPE=Debug
make
```

You may look into `CMakeLists.txt`. Remove `CMakeCache.txt` and `CMakeFiles/` in advance if you want a clean build.

- Compiler bug: Sometimes, compilers have trouble with the `-O2` optimization. You can often avoid such bugs with lower optimization options (`-O1` or `-O0`), depending on the source files, as described in `CMakeLists.txt`. Usually, people do not use `-O3`. You may set such conditional compilation settings. See `CMakeLists.txt`.
- Source codes, tests, and make system are under `SRC/`:

```
SRC/
├─ TestInstall : Root of install test
├─ exec       : CMakeLists.txt and scripts
├─ main       : All main *.f90
└─ subroutines : All subroutines *.f90
```

All Fortran codes are in `main/` and `subroutines/`.

`CMakeLists.txt` generates the Makefile with `cmake`.

- Test system: The install test system is at `ecalj/SRC/TestInstall/`.

Look into `test.py` and `testecalj.py`. These control all the tests. It is not difficult to add your own test to `testecalj.py`:

- Compute something at first.
- Store inputs and minimum results in a directory.
- Describe the comparison check in `testecalj.py`.

To test all binaries:

```
./test.py                # all tests
./test.py gwall          # tests only GW part
./test.py si_gwsc nio_gwsc # test si_gwsc and nio_gwsc only
```

- Note: On Ubuntu 22, I observed that `gfortran` + `openmpi` sometimes failed. Use `mpich` if you encounter issues. (Current status unknown.)